

Grover 搜索算法可视化演示系统课程报告

学生姓名：_____ 学号：_____

2026 年 7 月 1 日

1 项目简介

本课程项目搭建了一个针对 2 到 3 个量子比特的 Grover 搜索算法玩具模型，面向课堂演示与原理理解，重点展示目标态概率幅在 Oracle 相位翻转和均值倒置（inversion about average）的交替作用下逐步被放大的过程。

系统将一次 Grover 迭代拆分为可逐步观察的中间过程：首先通过 Hadamard 门制备均匀叠加态，使所有计算基态具有相同的初始概率幅；随后由 Oracle 对目标态执行相位翻转，使目标态概率幅由正值变为负值；最后由 Diffusion 算子完成关于平均概率幅的反演，将 Oracle 产生的相位差转化为目标态概率幅的增大。程序在 Hadamard 初始化、每一次 Oracle 翻转以及每一次均值倒置操作后保存量子态向量（statevector），并通过条形图（Bar Chart）、目标态概率变化曲线和 3D 步进柱状图展示各计算基态概率幅与概率的变化。

在实现上，项目使用 Qiskit 构造 Grover 量子线路并提取中间态，使用 Streamlit 搭建交互式界面，使用 Matplotlib 和 Plotly 完成 2D 与 3D 可视化。用户可以选择量子比特数、目标态和迭代次数，并通过滑块逐步观察每个操作之后的状态变化。例如，在 3 量子比特、目标态为 $|101\rangle$ 的演示中，界面会依次展示初始化、Oracle 翻转和均值倒置后的柱状图变化，从而直观说明 Grover 算法通过相位标记与量子干涉逐步放大目标态概率幅的过程。

2 背景与目标

Grover 搜索算法是量子计算中具有代表性的量子搜索算法，由 Lov Grover 于 1996 年提出，常用于说明量子算法在无结构搜索问题中的平方级加速能力。对于一个没有额外排序或索引结构的搜索空间，经典算法通常需要逐个检查候选项；Grover 算法可以在量子模型下将查询次数由 $O(N)$ 降低到 $O(\sqrt{N})$ 。因此，它既是量子算法课程中的基础内容，也是理解量子叠加、相位干涉和振幅放大的重要例子。

在一般搜索问题中，若数据库中有 N_{data} 个候选条目，并使用 n 个量子比特表示候选索引，则量子计算空间大小为 $N = 2^n$ 。实际情况下可能有 $N \geq N_{data}$ ，多出的计算基

态需要通过额外约束处理。为突出算法机制并简化实现，本项目采用课程演示中更常见的理想情况，即 $N = 2^n = N_{data}$ ，每一个计算基态都对应一个有效候选项。

根据课程项目要求，本项目搭建一个针对 2 到 3 个量子比特的 Grover 搜索算法玩具模型，并使用条形图或 3D 柱状图，以步进式（step-by-step）的方式展示每一次 Oracle 翻转和每一次均值倒置操作之后，各计算基态概率幅尤其是目标态概率幅的变化过程。项目将重点放在 Grover 算法的内部动态过程，即目标态概率幅如何经历相位翻转、均值倒置和逐步放大。

从算法流程看，Grover 算法可以理解为“相位标记”和“振幅放大”的组合。算法首先用 Hadamard 门将所有候选索引制备为均匀叠加态；随后用 Oracle 根据判定函数 $f(x)$ 标记目标解；最后通过 Diffusion 算子执行关于平均概率幅的反演。Oracle 把目标态概率幅乘以 -1 ，使目标态与非目标态之间产生相对相位差；后续的均值倒置操作再将这种相位差转化为目标态概率幅的增大。

基于上述特点，本项目将判定函数进一步简化为“当前索引是否等于用户选择的目标比特串”。用户直接选择一个计算基态作为目标态，例如 $|10\rangle$ 或 $|101\rangle$ 。这种设计弱化了实际问题中 Oracle 构造的复杂性，保留了 Grover 算法中相位翻转、均值倒置和振幅放大的核心过程，以展示概率幅如何随步骤逐步变化。

因此，根据课程项目要求，本项目的目标设定为：

1. 构造一个支持 2 到 3 个量子比特的 Grover 搜索算法玩具模型；
2. 支持用户选择任意计算基目标态，例如 $|10\rangle$ 、 $|101\rangle$ 等；
3. 在 Hadamard 初始化、每一次 Oracle 相位翻转和每一次均值倒置操作后保存中间量子态向量（statevector）；
4. 使用条形图（Bar Chart）展示每一步之后各计算基态的概率幅或概率分布；
5. 使用 3D 步进柱状图展示不同步骤之间概率幅分布的连续变化，突出目标态概率幅被逐步放大的过程；
6. 通过交互式模块、电路视图和目标态概率曲线辅助比较 Oracle 翻转与均值倒置的不同作用。

3 基本原理

3.1 问题模型

设搜索空间大小为

$$N = 2^n,$$

其中 n 为量子比特数。本项目只考虑 $n = 2$ 或 $n = 3$ 的玩具模型，因此搜索空间分别包含 4 个或 8 个计算基态。单目标搜索中，目标索引记为 ω ，对应的目标态为 $|\omega\rangle$ 。算法从初始状态 $|0\rangle^{\otimes n}$ 出发，首先对所有量子比特施加 Hadamard 门，将系统制备为均匀叠加态：

$$|s\rangle = H^{\otimes n}|0\rangle^{\otimes n} = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle.$$

该状态中，每一个计算基态都具有相同的初始概率幅 $1/\sqrt{N}$ 。因此，如果此时直接测量，得到目标态 $|\omega\rangle$ 的概率为

$$P_0 = \left| \frac{1}{\sqrt{N}} \right|^2 = \frac{1}{N}.$$

当 $n = 3$ 时， $N = 8$ ，目标态初始概率只有 12.5%。Grover 算法通过相位标记和量子干涉逐步提高目标态的概率幅与测量概率。

3.2 Oracle 相位标记

为了描述目标态的识别过程，通常引入判定函数

$$f(x) = \begin{cases} 1, & x = \omega, \\ 0, & x \neq \omega. \end{cases}$$

在真实应用中， ω 通常由具体问题的约束条件间接确定。例如， $f(x)$ 可以表示某个候选密钥是否正确，或某个候选解是否满足约束。本项目将问题简化为由用户直接选择 ω ，如 $|10\rangle$ 、 $|101\rangle$ 等。

Oracle 的作用是根据 $f(x)$ 对目标态进行相位标记。如果引入辅助量子比特，Oracle 可以写成可逆变换

$$|x\rangle|q\rangle \mapsto |x\rangle|q \oplus f(x)\rangle.$$

当辅助量子比特取为

$$|-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

时，由于 $X|-\rangle = -|-\rangle$ ，上述操作会通过相位反冲得到等效的相位 Oracle：

$$O|x\rangle = (-1)^{f(x)}|x\rangle.$$

因此，对于单目标情形有

$$O|\omega\rangle = -|\omega\rangle, \quad O|x\rangle = |x\rangle \quad (x \neq \omega).$$

Oracle 改变目标态概率幅的符号，并保持其测量概率不变。若目标态概率幅为 a_ω ，经过 Oracle 后变为 $-a_\omega$ ，概率仍为

$$P_\omega = |a_\omega|^2 = |-a_\omega|^2.$$

因此，在概率幅条形图中可以看到目标态柱子从零轴上方翻转到零轴下方；在概率图中，目标态柱子的高度暂时保持不变。

3.3 Diffusion 均值倒置

Diffusion 算子定义为

$$D = 2|s\rangle\langle s| - I,$$

其中 $|s\rangle$ 是 Hadamard 初始化后得到的均匀叠加态。由于

$$|s\rangle = H^{\otimes n}|0^n\rangle,$$

其中 $|0^n\rangle = |0\rangle^{\otimes n}$, 因此 Diffusion 算子也可以写成

$$D = H^{\otimes n} (2|0^n\rangle\langle 0^n| - I) H^{\otimes n}.$$

这说明在量子线路中, 可以先用 Hadamard 门把 $|s\rangle$ 方向变换回 $|0^n\rangle$ 方向, 再对 $|0^n\rangle$ 做选择性相位操作, 最后再通过 Hadamard 门变回原来的叠加基底。

从概率幅角度看, Diffusion 算子等价于把每个概率幅 a_i 关于平均概率幅 $\bar{a}$ 做反演。设当前量子态为

$$|\psi\rangle = \sum_{i=0}^{N-1} a_i |i\rangle,$$

其中平均概率幅为

$$\bar{a} = \frac{1}{N} \sum_{i=0}^{N-1} a_i.$$

则 Diffusion 作用后, 每个概率幅变为

$$a'_i = 2\bar{a} - a_i.$$

Oracle 先把目标态振幅翻成负值, 使目标态振幅低于平均值; Diffusion 再对所有振幅做均值倒置, 于是目标态振幅从负值被反演到较大的正值, 非目标态振幅被相应压低。这样, Oracle 制造的相位信息被转化为目标态概率幅的放大。在本项目的步进式可视化中, Oracle 之后的图像用于观察目标态概率幅符号翻转, Diffusion 之后的图像用于观察目标态概率幅变大。

3.4 几何解释

除了从概率幅角度理解 Grover 算法, 还可以把它看作二维状态空间中的几何旋转。对于单目标搜索, 定义目标方向为

$$|\beta\rangle = |\omega\rangle,$$

非目标方向为所有非目标计算基态的归一化叠加:

$$|\alpha\rangle = \frac{1}{\sqrt{N-1}} \sum_{x \neq \omega} |x\rangle.$$

由于目标态 $|\omega\rangle$ 与所有非目标计算基态互不相同, 而计算基态之间满足正交关系 $\langle x|y\rangle = \delta_{xy}$, 因此有

$$\langle \alpha|\beta\rangle = 0.$$

也就是说, $|\alpha\rangle$ 和 $|\beta\rangle$ 可以看作二维平面中的两条正交坐标轴。

在这组基底下, 初始均匀叠加态可以分解为

$$|s\rangle = \sqrt{\frac{N-1}{N}}|\alpha\rangle + \sqrt{\frac{1}{N}}|\beta\rangle.$$

为了描述 $|s\rangle$ 在该二维平面中的方向, 引入角度 θ , 使得

$$\cos \theta = \sqrt{\frac{N-1}{N}}, \quad \sin \theta = \frac{1}{\sqrt{N}}.$$

于是

$$|s\rangle = \cos \theta |\alpha\rangle + \sin \theta |\beta\rangle.$$

这说明 θ 就是初始态 $|s\rangle$ 相对于非目标方向 $|\alpha\rangle$ 的夹角。由于 $1/\sqrt{N}$ 通常很小, 初始态一开始更接近非目标方向, 目标方向上的分量较小。

在该二维平面中, Oracle 使目标方向 $|\beta\rangle$ 的符号翻转, 并保持非目标方向 $|\alpha\rangle$ 不变, 因此它等价于关于 $|\alpha\rangle$ 轴的一次反射。Diffusion 算子 $D = 2|s\rangle\langle s| - I$ 保持 $|s\rangle$ 方向不变, 并翻转与 $|s\rangle$ 正交的分量, 因此等价于关于 $|s\rangle$ 方向的一次反射。平面几何中, 关于两条夹角为 θ 的直线连续反射, 等价于旋转 2θ 。所以一次完整 Grover 迭代

$$G = DO$$

会使量子态在 $|\alpha\rangle$ 和 $|\beta\rangle$ 张成的二维平面内向目标方向旋转 2θ 。

3.5 迭代次数与概率变化

上述几何解释可以直接导出 Grover 算法的概率变化公式。由于初始态与 $|\alpha\rangle$ 方向的夹角为 θ , 而每一次完整 Grover 迭代都会额外旋转 2θ , 因此完成 r 次迭代后, 状态方向与 $|\alpha\rangle$ 的夹角为

$$(2r+1)\theta.$$

于是系统状态可以写成

$$|\psi_r\rangle = \cos((2r+1)\theta)|\alpha\rangle + \sin((2r+1)\theta)|\beta\rangle.$$

其中 $|\beta\rangle$ 就是目标方向, 因此测量得到目标态的理论概率为

$$P_r = \sin^2((2r+1)\theta).$$

为了使目标态概率尽可能接近 1, 需要令

$$(2r+1)\theta \approx \frac{\pi}{2}.$$

因此，单目标情况下的最优迭代次数近似为

$$r^* \approx \left\lfloor \frac{\pi}{4\theta} - \frac{1}{2} \right\rfloor.$$

当 N 较大时， $\sin \theta = 1/\sqrt{N}$ ，因此 $\theta \approx 1/\sqrt{N}$ ，从而得到常见近似式

$$r^* \approx \left\lfloor \frac{\pi}{4} \sqrt{N} \right\rfloor.$$

如果推广到 M 个目标解，则有

$$\sin \theta = \sqrt{\frac{M}{N}},$$

对应的迭代次数近似为

$$r^* \approx \left\lfloor \frac{\pi}{4} \sqrt{\frac{N}{M}} \right\rfloor.$$

Grover 迭代次数存在最优范围。由于目标态概率满足

$$P_r = \sin^2((2r+1)\theta),$$

而 $\sin^2(\cdot)$ 是周期函数，当状态向量旋转到接近目标方向后，继续迭代会使其转过目标方向，导致目标态概率重新下降。这就是 Grover 算法中的“过迭代”现象。由于本项目只展示 2 到 3 个量子比特，搜索空间较小，且主要演示 $M=1$ 的单目标搜索，目标态概率会在少数几次迭代内迅速升高；界面中保留迭代次数选择和目标态概率曲线，用于辅助观察这种先上升后回落的周期性变化。

4 系统设计与实现

4.1 项目结构

本项目代码结构如下：

```
grover_qiskit_ui_refined/
|-- app.py                # Streamlit交互界面
|-- main.py               # 命令行图片和CSV导出入口
|-- qiskit_grover_core.py # Grover电路构造与statevector分析
|-- visualization.py      # Matplotlib/Plotly可视化函数
|-- requirements.txt      # Python依赖
`-- README.md             # 项目说明
```

其中，`qiskit_grover_core.py` 负责算法和量子态演化，`visualization.py` 负责图形绘制，`app.py` 负责交互界面组织，`main.py` 用于离线生成报告图片和 CSV 数据。

4.2 主要工具

项目采用的主要工具如下：

表 1: 项目技术选型

模块	工具	作用
量子电路	Qiskit	构造 Hadamard、Oracle、Diffusion 和测量电路
中间态分析	Statevector	提取每一步后的精确量子态
静态绘图	Matplotlib	导出概率幅图、概率图和标准电路图
交互绘图	Plotly	绘制交互式柱状图、3D 概率历史图和动画
网页界面	Streamlit	提供参数选择、滑块和图表展示
可选仿真	Qiskit Aer	进行 shots 测量统计和简化噪声对照

4.3 算法流程与状态记录

本项目把一次 Grover 搜索拆分为 Hadamard 初始化、Oracle 相位翻转和 Diffusion 均值倒置三个核心阶段。程序在每个关键操作后保存当前量子态，并把这些中间状态作为后续图表的数据来源。

核心流程为：

1. 初始化 n 个量子比特为 $|0\rangle^{\otimes n}$ ；
2. 对所有量子比特施加 Hadamard 门，得到均匀叠加态，并保存初始 statevector；
3. 每轮 Grover 迭代先施加 Oracle，对目标态执行相位翻转，并保存 Oracle 后的 statevector；
4. 再施加 Diffusion 算子，完成关于平均概率幅的反演，并保存 Diffusion 后的 statevector；
5. 将所有中间状态传给可视化模块，绘制概率幅柱状图、概率柱状图和 3D 步进图。

程序定义了 GroverStep 数据结构，用于记录每一个中间阶段：

- index: 步骤编号；
- kind: 步骤类型，包括 initial、oracle、diffusion；
- iteration: 当前属于第几轮 Grover 迭代；
- circuit: 当前量子电路副本；
- statevector: 当前电路对应的精确量子态。

4.4 运行方式

项目提供两种运行入口。若需要启动交互式界面，可在项目根目录执行：

```
streamlit run app.py
```

界面启动后，用户可以选择量子比特数、目标态和 Grover 迭代次数，程序会实时生成概率幅图、概率图、目标态概率曲线和 3D 步进图。

若需要生成报告中使用的图片和 CSV 数据，可在命令行中直接指定目标量子态和迭代次数：

```
python main.py --n 3 --target 101 --iterations 2
```

其中，`--n` 表示量子比特数，`--target` 表示目标态比特串，`--iterations` 表示 Grover 迭代次数。程序根据这些参数构造量子电路，保存每一步 `statevector`，并导出对应图像和数据文件。

4.5 关键实现方法

4.5.1 多控制相位翻转

Oracle 和 Diffusion 都需要对 $|11\cdots 1\rangle$ 执行相位翻转。程序将这个操作封装为 `_apply_multi_controlled_z`。当量子比特数为 1 时直接使用 Z 门；当量子比特数大于 1 时，程序用 $H - \text{MCX} - H$ 把多控制 X 门转化为多控制 Z 门：

```
def _apply_multi_controlled_z(qc, qubits):
    if len(qubits) == 1:
        qc.z(qubits[0])
        return

    controls = qubits[:-1]
    target = qubits[-1]
    qc.h(target)
    qc.mcx(controls, target)
    qc.h(target)
```

以 3 量子比特为例，该函数会对 $|111\rangle$ 态乘以 -1 ，其余计算基态保持不变。

4.5.2 目标态 Oracle 构造

项目允许用户选择任意目标态。Oracle 的目标是只对该目标态执行相位翻转。实现时，程序先把目标态临时映射到 $|11\cdots 1\rangle$ ，再调用多控制相位翻转，最后恢复原来的计算基顺序。

例如目标态为 $|101\rangle$ 时，中间的第 2 位为 0，程序会先对对应量子比特施加 X 门，把 $|101\rangle$ 映射到 $|111\rangle$ 。完成多控制 Z 门后，再施加一次 X 门还原：


```
def apply_oracle(qc, target, add_barrier=True):
    n = len(target)

    for qubit, bit in enumerate(target):
        if bit == "0":
            qc.x(qubit)

    _apply_multi_controlled_z(qc, list(range(n)))

    for qubit, bit in enumerate(target):
        if bit == "0":
            qc.x(qubit)

    if add_barrier:
        qc.barrier()
```

因此，对于任意用户输入的目标比特串，Oracle 都会实现

$$O|\omega\rangle = -|\omega\rangle, \quad O|x\rangle = |x\rangle \quad (x \neq \omega).$$

4.5.3 Diffusion 算子构造

Diffusion 算子采用标准的 $H - X - \text{MCZ} - X - H$ 结构。程序先用 Hadamard 门和 X 门把关于均匀叠加态的反演转换为对 $|11 \cdots 1\rangle$ 的相位翻转，再恢复基底：

```
def apply_diffusion(qc, n, add_barrier=True):
    for qubit in range(n):
        qc.h(qubit)
    for qubit in range(n):
        qc.x(qubit)

    _apply_multi_controlled_z(qc, list(range(n)))

    for qubit in range(n):
        qc.x(qubit)
    for qubit in range(n):
        qc.h(qubit)

    qc.global_phase += np.pi

    if add_barrier:
        qc.barrier()
```

其中, `qc.global_phase += np.pi` 用于校正全局相位。经过该处理后, 保存的 `statevector` 与

$$a'_i = 2\bar{a} - a_i$$

一致, Diffusion 后的概率幅可以直接解释为关于平均概率幅的反演。

4.5.4 完整 Grover 电路构造

完整电路由 Hadamard 初始化和若干轮 Grover 迭代组成。程序在 `build_grover_circuit` 中先创建量子电路, 再对全部量子比特施加 Hadamard 门, 随后按迭代次数重复调用 Oracle 和 Diffusion:

```
def build_grover_circuit(n, target, iterations,
                        measure=False, add_barriers=True):
    validate_problem(n, target, iterations)

    if measure:
        qc = QuantumCircuit(n, n)
    else:
        qc = QuantumCircuit(n)

    qc.h(range(n))
    if add_barriers:
        qc.barrier()

    for _ in range(iterations):
        apply_oracle(qc, target, add_barrier=add_barriers)
        apply_diffusion(qc, n, add_barrier=add_barriers)

    if measure:
        for qubit in range(n):
            qc.measure(qubit, n - 1 - qubit)

    return qc
```

测量模式下, 程序把第 i 个量子比特测量到第 $n - 1 - i$ 个经典比特, 使最终输出的比特串顺序与用户输入的目标态顺序一致。

4.5.5 中间态保存

为了得到每一步后的概率幅数据, 程序没有只保存最终电路, 而是在 `build_grover_history` 中逐步构造电路。每完成 Hadamard 初始化、Oracle 或 Diffusion 操作, 就复制当前电路, 并用 `Statevector.from_instruction(qc)` 计算当前量子态:

```
def build_grover_history(n, target, iterations):
    validate_problem(n, target, iterations)

    qc = QuantumCircuit(n)
    history = []

    qc.h(range(n))
    qc.barrier()
    history.append(GroverStep(
        index=0,
        kind="initial",
        iteration=0,
        circuit=qc.copy(),
        statevector=Statevector.from_instruction(qc),
    ))

    step_index = 1
    for iteration in range(1, iterations + 1):
        apply_oracle(qc, target, add_barrier=True)
        history.append(GroverStep(
            index=step_index,
            kind="oracle",
            iteration=iteration,
            circuit=qc.copy(),
            statevector=Statevector.from_instruction(qc),
        ))
        step_index += 1

        apply_diffusion(qc, n, add_barrier=True)
        history.append(GroverStep(
            index=step_index,
            kind="diffusion",
            iteration=iteration,
            circuit=qc.copy(),
            statevector=Statevector.from_instruction(qc),
        ))
        step_index += 1

    return history
```

该历史序列直接作为后续概率幅图、概率图和 3D 步进图的数据来源。

5 可视化效果

5.1 交互参数

Streamlit 界面左侧栏提供如下参数：

- 量子比特数：2 或 3；
- 目标态：从全部计算基态中选择；
- 最大 Grover 迭代次数；
- 步进动画间隔；
- 是否显示 statevector 表格、Qiskit 电路图、理想/带噪声实验对照。

主页面按观察顺序组织为：

1. 当前步骤的概率幅图和概率图；
2. 目标态概率随完整迭代次数变化曲线；
3. Qiskit 标准电路图与交互式门操作视图；
4. 理想概率和带噪声测量频率对照；
5. 理论公式与过迭代现象解释。

6 运行结果

6.1 3 量子比特目标态 $|101\rangle$ 示例

选取 $n = 3$ 、目标态 $|101\rangle$ 、迭代次数为 2。此时搜索空间大小为 $N = 8$ ，初始每个基态概率幅为

$$\frac{1}{\sqrt{8}} \approx 0.353553,$$

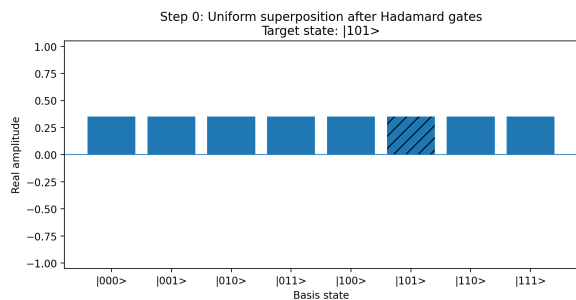
每个基态概率为 $1/8 = 0.125$ 。

表 2: $n = 3$ 、目标态 $|101\rangle$ 时目标态概率幅和概率变化

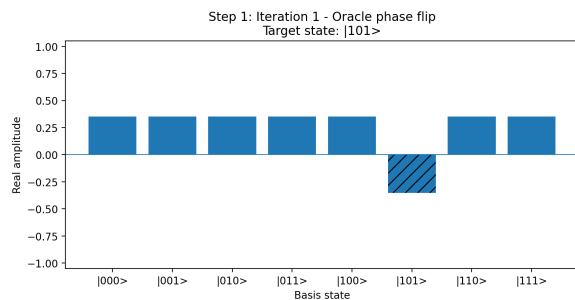
步骤	操作	目标态概率幅	目标态概率
S0	Hadamard 均匀叠加	+0.353553	0.125000
S1	第 1 次 Oracle 相位翻转	-0.353553	0.125000
S2	第 1 次 Diffusion 均值倒置	+0.883883	0.781250
S3	第 2 次 Oracle 相位翻转	-0.883883	0.781250
S4	第 2 次 Diffusion 均值倒置	+0.972272	0.945312

从表中可以看出：

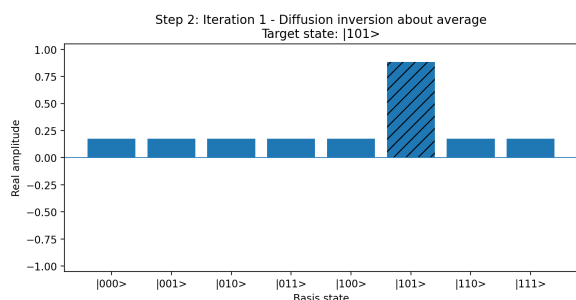
- Oracle 步骤只改变目标态概率幅符号，不改变目标态概率；
- Diffusion 步骤把目标态概率幅大幅推高；
- 两次完整迭代后，目标态概率达到约 94.53%；
- 如果继续迭代，目标概率会越过最佳点并下降，因此 Grover 算法需要选择合适的迭代次数。



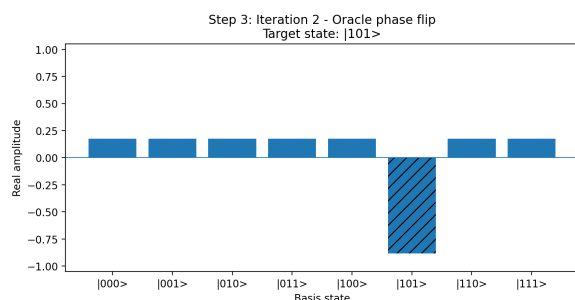
(a) 初始均匀叠加态的概率幅分布



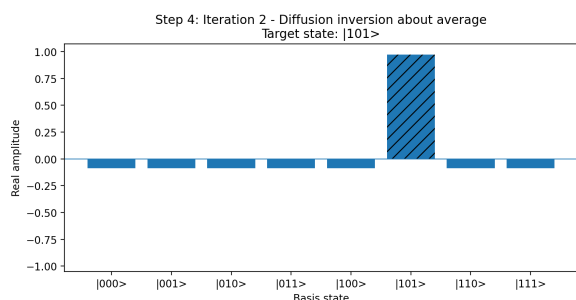
(b) 第 1 次 Oracle 后目标态概率幅被翻转



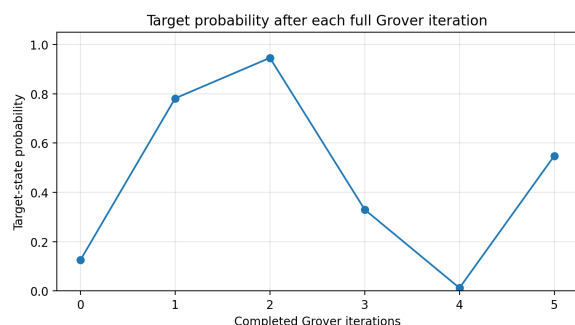
(c) 第 1 次 Diffusion 后目标态概率幅被放大



(d) 第 2 次 Oracle 后目标态概率幅再次翻转



(e) 第 2 次 Diffusion 后目标态概率幅继续增大



(f) 目标态概率随完整 Grover 迭代次数变化

图 1: $n = 3$ 、目标态 $|101\rangle$ 时两次 Grover 迭代中的概率幅变化

6.2 与理论结果对照

对 $N = 8$, 有

$$\theta = \arcsin\left(\frac{1}{\sqrt{8}}\right).$$

代入理论公式:

$$P_1 = \sin^2(3\theta) = 0.78125,$$

$$P_2 = \sin^2(5\theta) = 0.9453125.$$

这与程序中 `statevector` 计算得到的结果一致，说明电路构造和图表数据提取是正确的。

6.3 2 量子比特情况

当 $n = 2$ 时， $N = 4$ 。单目标 Grover 搜索在一次完整迭代后即可把目标态概率提升到 1。当 $n = 3$ 时，目标态概率在多次迭代中先上升后回落，程序通过目标态概率曲线记录这一过迭代现象。

7 问题与改进

7.1 已完成的功能

1. **保存中间态。**项目在每次 Oracle 和 Diffusion 后保存 `statevector`，形成完整的步骤序列。
2. **同时计算概率幅和概率。**概率幅图给出 Oracle 后的符号翻转，概率图给出 Diffusion 后的测量概率提升。
3. **生成 3D 步进图。**3D 图把“基态-步骤-概率”放在同一个坐标系中，记录目标态概率柱随步骤升高的过程。
4. **提供交互参数。**用户可以切换量子比特数、目标态、迭代次数，并通过滑块选择当前步骤。
5. **保留量子电路信息。**除图表外，界面还显示 Qiskit 标准电路图和交互式操作表，使每一步状态变化对应到具体量子门。

7.2 不足与改进方向

本项目按照课程项目要求支持 2 到 3 个量子比特。若扩展到更多量子比特，柱状图数量会迅速增加，3D 图也会变得拥挤，需要改用筛选、聚类或只突出目标态和若干代表性非目标态的方式。

另一方面，当前噪声模型只是简化的 depolarizing noise 和 readout noise，不代表真实量子硬件的校准噪声。后续可以接入真实后端参数，或加入多目标 Grover 搜索。

8 总结

本项目完成了一个基于 Qiskit、Streamlit 和 Plotly 的 Grover 搜索算法振幅放大可视化系统。系统支持 2 到 3 个量子比特，能够选择任意目标态，并在 Hadamard 初始化、Oracle 相位翻转和 Diffusion 均值倒置后保存中间 `statevector`。

通过 $n = 3$ 、目标态 $|101\rangle$ 的实验可以看到, Oracle 只是将目标态概率幅变为负值, 目标态概率并未立即增加; 随后 Diffusion 通过关于平均概率幅的反演, 把相位标记转化为振幅放大, 使目标态概率从 12.5% 逐步提升到 78.125% 和 94.53125%。这一结果与 Grover 算法理论公式一致。

总体来看, 本项目完成了课程报告所需的主要内容: 实现了一个可运行的 Grover 算法程序, 说明了核心算法步骤, 并用图表记录了目标态概率被逐步放大的过程。程序结果表明, Oracle 相位标记和 Diffusion 均值倒置在 Grover 算法中承担了不同作用。

参考文献

- [1] L. K. Grover, “A fast quantum mechanical algorithm for database search,” Proceedings of the 28th Annual ACM Symposium on Theory of Computing, 1996.
- [2] Qiskit Documentation, <https://docs.quantum.ibm.com/>
- [3] Streamlit Documentation, <https://docs.streamlit.io/>
- [4] Plotly Python Documentation, <https://plotly.com/python/>